Chapter 15 | Exercise solutions with Python

Delta Rules and Hybrid Stacks

This notebook solves Exercises 5–8. It reads a delta state with two queries, traces a hybrid stack and its persistent stores, and checks an attention cache against a full causal pass. Exercises 1–4 and 9–10 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch15_hybrids"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check a read with Python

Imagine the two columns of the state as two stored values. A read with the first coordinate vector picks the first column, and a read with the second picks the second column. The **key** says which direction to correct. The **query** says which direction to read after that correction. Here we write in one direction and try both reads.

Start with the matrix S from Exercise 1. Apply the decay $\alpha = 1/2$ to every entry before predicting the value at the key $k = (0, 1)^\top$. The decayed matrix and its prediction are

$$S^- = \frac{1}{2}S = \begin{bmatrix} 1/2 & 1 \\ 0 & 3/2 \end{bmatrix}, \qquad S^- k = \begin{bmatrix} 1 \\ 3/2 \end{bmatrix}.$$

Subtract the prediction from the desired value $v = (4, -1)^\top$. The error is $e = (3, -5/2)^\top$. The outer product $ek^\top$ places a copy of the error in the second column and zeros in the first. With a correction strength of $\beta = 1/4$, the state becomes

$$S^{+} = S^{-} + \frac{1}{4} e k^{\top} = \begin{bmatrix} 1/2 & 1 \\ 0 & 3/2 \end{bmatrix} + \begin{bmatrix} 0 & 3/4 \\ 0 & -5/8 \end{bmatrix} = \begin{bmatrix} 1/2 & 7/4 \\ 0 & 7/8 \end{bmatrix}.$$

The first column changes from $(1, 0)^{\top}$ to $(1/2, 0)^{\top}$ by decay alone. The second column moves one quarter of the way from the decayed prediction toward the desired value. Compare both reads with the hand results. Each call starts from a copy of the same original matrix. Reusing the first returned state would apply a second write and change the experiment.

```
from examples.ch15_hybrids.model import delta_step

state_original = np.array([[1.0, 2.0], [0.0, 3.0]])
read_key = np.array([0.0, 1.0])
read_value = np.array([4.0, -1.0])
read_second = np.array([0.0, 1.0])
read_first = np.array([1.0, 0.0])
decay, correction = 0.5, 0.25

output_second, state_second = delta_step(
    state_original.copy(), read_second, read_key, read_value,
    decay, correction,
)
output_first, state_first = delta_step(
    state_original.copy(), read_first, read_key, read_value,
    decay, correction,
)
```

For this particular state, the first basis query reads $(1/2, 0)^{\top}$ and the second reads $(7/4, 7/8)^{\top}$. The terms in the update use the key and value, with no query term. The final multiplication uses the query. Check the shared matrix and the individual readouts in float64 with absolute tolerance 10^{-12} and zero relative tolerance. Both results here use fractions whose denominators are powers of two, which float64 can represent exactly at these magnitudes.

```
read_atol = 1e-12
state_expected = np.array([[0.5, 1.75], [0.0, 0.875]])
for observed_state in (state_second, state_first):
    np.testing.assert_allclose(
        observed_state, state_expected, atol=read_atol, rtol=0,
    )
np.testing.assert_array_equal(state_second, state_first)
for output, expected_column in (
    (output_second, [1.75, 0.875]), (output_first, [0.5, 0.0]),
):
    np.testing.assert_allclose(
        output, expected_column, atol=read_atol, rtol=0,
    )

print("Updated state for either query")
print(state_second)
print("Second basis query reads", output_second)
print("First basis query reads ", output_first)
```

```
OUTPUT
Updated state for either query
[[0.5   1.75 ]
 [0.    0.875]]
Second basis query reads [1.75  0.875]
First basis query reads  [0.5 0. ]
```

6. Watch three stores grow

Follow one token at a time through three layers. Layers 1 and 3 use delta updates and each carries its own 2×2 matrix state. Layer 2 uses full attention and carries two arrays, one for keys and one for values. Both arrays start with zero rows and have feature width two. Each token adds one row to each array. After t tokens, the shapes are $(t, 2)$ and $(t, 2)$, with $2t$ key scalars and $2t$ value scalars.

Each delta state represents the accumulated effects of its layer's writes in the same four entries. Its entries can change at every step. Its shape stays fixed. The attention layer keeps one key and one value vector per token, so its history grows by four scalars per step. The expected layer counts are therefore $(4, 4t, 4)$. The total includes both separate delta matrices and both parts of the attention cache, giving $8 + 4t$ scalars.

After reading	Layer 1 delta	Layer 2 keys	Layer 2 values	Layer 3 delta	Total scalars
No tokens	4	0	0	4	8
$(1, 2)$	4	2	2	4	12
$(0, 1)$	4	4	4	4	16
$(-1, 2)$	4	6	6	4	20

The second layer receives the representation produced by layer 1 at this step. The projection rule makes this incoming representation the new key and swaps its two coordinates to form the new value. The attention cache therefore stores the first layer's transformed features. Check the newest row in each array against that rule.

```
from examples.ch15_hybrids.model import (
    history_counts, initial_stores, stack_step,
)

hybrid_schedule = ("delta", "attention", "delta")
hybrid_tokens = np.array([[1.0, 2.0], [0.0, 1.0], [-1.0, 2.0]])
stores = initial_stores(hybrid_schedule)
assert history_counts(stores) == (4, 0, 4)

print("step  layer 1    keys    values    layer 3   per layer")
for t, token in enumerate(hybrid_tokens, start=1):
    trace, stores = stack_step(token, hybrid_schedule, stores)
    left_state, (keys, values), right_state = stores
    assert left_state.shape == right_state.shape == (2, 2)
    assert keys.shape == values.shape == (t, 2)
    np.testing.assert_array_equal(keys[-1], trace[1])
    np.testing.assert_array_equal(values[-1], trace[1][::-1])
    counts = history_counts(stores)
    assert counts == (4, 4 * t, 4)
    assert all(a.dtype == np.float64 for a in (
        left_state, keys, values, right_state,
    ))
    print(f"{t:>4}  {str(left_state.shape):>7}  "
          f"{str(keys.shape):>6}  {str(values.shape):>8}  "
          f"{str(right_state.shape):>9}   {counts}")
```

```
OUTPUT
step  layer 1    keys    values    layer 3   per layer
   1   (2, 2)  (1, 2)    (1, 2)     (2, 2)   (4, 4, 4)
   2   (2, 2)  (2, 2)    (2, 2)     (2, 2)   (4, 8, 4)
   3   (2, 2)  (3, 2)    (3, 2)     (2, 2)   (4, 12, 4)
```

Each iteration passes forward the **returned** stores. The input and intermediate representations in `trace` let us inspect this step. They are not additional persistent history. The old state and old cache objects are replaced by their new versions, so adding the counts from successive steps would double-count retained history. With eight bytes per float64 scalar, the three measured totals after inputs 1, 2, and 3 are 96, 128, and 160 bytes. Those numbers count the array payload only and exclude parameters, temporary attention work, and Python object overhead.

7. Change the middle mixer

Feed the same sequence $(1, 2)$, $(0, 1)$, $(-1, 2)$ through two three-layer stacks. The hybrid has delta, attention, and delta mixers. The other has three delta mixers. Each run starts with its own fresh stores and passes the returned stores to the next token. Each mixer's output becomes the next mixer's input for the same token.

For every token, `trace[0]` is the original representation. Entries `trace[1]`, `trace[2]`, and `trace[3]` are the outputs of the three successive mixers. Compare the first- and third-layer outputs for the final token. Also inspect the middle output and the last layer's state just before that token to see where the difference comes from.

```
from examples.ch15_hybrids.model import initial_stores, stack_step

schedule_tokens = np.array(
    [[1, 2], [0, 1], [-1, 2]], dtype=np.float64
)
hybrid_schedule = ("delta", "attention", "delta")
delta_schedule = ("delta", "delta", "delta")
schedule_atol = 1e-12

def trace_schedule(schedule):
    stores = initial_stores(schedule)
    traces = []
    last_layer_states = []
    for token in schedule_tokens:
        trace, stores = stack_step(token, schedule, stores)
        traces.append(np.stack(trace))
        last_layer_states.append(stores[-1].copy())
    return np.stack(traces), np.stack(last_layer_states)

hybrid_trace, hybrid_last_states = trace_schedule(hybrid_schedule)
delta_trace, delta_last_states = trace_schedule(delta_schedule)

print("Final token, entries 0 through 3 in each trace")
with np.printoptions(precision=9, suppress=True):
    for i in range(4):
        print(f"{i}  hybrid {hybrid_trace[-1, i]}  "
              f"all-delta {delta_trace[-1, i]}")
```

OUTPUT

```
Final token, entries 0 through 3 in each trace
0  hybrid [-1.  2.]  all-delta [-1.  2.]
1  hybrid [ 1.223955098 -0.471875   ]  all-delta [ 1.223955098 -0.471875   ]
2  hybrid [-0.035559528  1.059354575]  all-delta [-0.190662069  0.78165406 ]
3  hybrid [0.748677465 0.071532371]  all-delta [ 0.511765365 -0.05517097 ]
```

The first mixer has the same update rule, initial state, and input history in both runs. In each case the query and key are the incoming vector normalized to unit length. The value swaps its two coordinates. The gates are $\alpha = 3/4$ and $\beta = 1/2$. The first layer therefore follows the same sequence of states and gives the same representation for every token.

We can compute entry 1 of the final trace by hand. Let $a = 1/\sqrt{5}$ and write S_1 and S_2 for the first layer's states after the first and second tokens. The first key is $(a, 2a)^\top$ and the value is $(2, 1)^\top$. From zero, the update is half their outer product. The second token uses key $(0, 1)^\top$, so after decay only column 2 receives a correction.

$$S_1 = a\begin{bmatrix} 1 & 2 \\ 1/2 & 1 \end{bmatrix}, \qquad S_2 = \begin{bmatrix} 3a/4 & 1/2 + 3a/4 \\ 3a/8 & 3a/8 \end{bmatrix}.$$

For the third token, the query and key are both $k = (-a, 2a)^\top$, and the value is $v = (2, -1)^\top$. The decayed prediction $p = (3/4)S_2 k$ is $(3a/4 + 9/80,\ 9/160)^\top$. Reading along the same unit key after the update gives $p + \frac{1}{2}(v - p) = \frac{1}{2}(p + v)$. The first layer's final output is therefore $(169/160 + 3a/8,\ -151/320)^\top$, approximately $(1.223955098,\ -0.471875)$.

```
first_expected = np.array([
    169 / 160 + 3 / (8 * np.sqrt(5)),
    -151 / 320,
])
np.testing.assert_array_equal(hybrid_trace[:, 1], delta_trace[:, 1])
np.testing.assert_allclose(
    hybrid_trace[-1, 1], first_expected,
    rtol=0, atol=schedule_atol,
)

print("Layer 3 state before the final token")
for name, trace, states in (
    ("hybrid", hybrid_trace, hybrid_last_states),
    ("all-delta", delta_trace, delta_last_states),
):
    incoming = trace[-1, 2]
    key = incoming / np.linalg.norm(incoming)
    prediction = (0.75 * states[-2]) @ key
    recovered = 0.5 * (prediction + incoming[::-1])
    np.testing.assert_allclose(
        trace[-1, 3], recovered, rtol=0, atol=schedule_atol,
    )
    with np.printoptions(precision=9, suppress=True):
        print(name)
        print(states[-2])
final_difference = hybrid_trace[-1, 3] - delta_trace[-1, 3]
print("Final layer difference, hybrid minus all-delta")
with np.printoptions(precision=9, suppress=True):
    print(final_difference)
```

OUTPUT

```
Layer 3 state before the final token
hybrid
[[0.264078288 0.593193751]
 [0.111645674 0.24204746 ]]
all-delta
[[0.141749596 0.366533414]
 [0.053401097 0.123259043]]
Final layer difference, hybrid minus all-delta
[0.2369121   0.126703341]
```

At entry 2, the hybrid's attention mixer reads from the three stored keys and values. The all-delta middle mixer instead reads its updated matrix state. The final token reaches layer 3 as about $(-0.03556, 1.05935)$ in the hybrid and $(-0.19066, 0.78165)$ in the all-delta stack. Their preceding tokens also produced different middle outputs, so layer 3 enters the final step with a different matrix in each run.

The two last mixers follow the same rule. Each decays its own preceding state, forms a new prediction along the normalized incoming direction, and corrects toward the swapped incoming value. The check above reconstructs that read from the saved preceding state and the current middle output. The final outputs are about $(0.74868, 0.07153)$ and $(0.51177, -0.05517)$, respectively. The last mixer type is the same, but both the information entering it now and the information it carried from earlier tokens have changed. The NumPy comparisons use float64 with absolute tolerance 10^{-12} and zero relative tolerance, except for the exact check between the identical first-layer traces.

8. Check the cache against a full pass (optional)

Use the one-layer schedule `("attention",)` and the three input rows $(1, 0)$, $(0, 1)$, and $(1, 1)$. In the teaching attention layer, the query and key are the input itself and the value swaps its two features. This gives $Q = K$ equal to the input matrix and value rows $(0, 1)$, $(1, 0)$, and $(1, 1)$. In a full pass we compute all the queries together. A Boolean lower-triangular mask permits key j for query i exactly when $j \le i$. The diagonal keeps each token's own key available.

The two-feature dot product is divided by $\sqrt{2}$. For a hand check, put $s = 1/\sqrt{2}$ and $r = e^{-s}$. Subtracting the largest allowed score from each row before exponentiating gives the following mixtures.

Query	Allowed key positions	Allowed scores	Output
1	1	(s)	$(0, 1)$
2	1, 2	$(0, s)$	$(1, r)/(1 + r)$
3	1, 2, 3	$(s, s, 2s)$	$(1 + r, 1 + r)/(1 + 2r)$

For example, query 3 has normalized weights $(r, r, 1)/(1 + 2r)$. Both output coordinates receive a contribution 1 from the last value and a contribution r from one of the other values before division.

```
from examples.ch04_attention.attention import (
    scaled_dot_product_attention,
)

cache_tokens = np.array([[1, 0], [0, 1], [1, 1]], dtype=np.float64)
full_queries = cache_tokens.copy()
full_keys = cache_tokens.copy()
full_values = cache_tokens[:, ::-1].copy()
allowed = np.tri(len(cache_tokens), dtype=bool)
full_output, full_weights = scaled_dot_product_attention(
    full_queries, full_keys, full_values, allowed_mask=allowed,
)

r = np.exp(-1 / np.sqrt(2))
hand_output = np.array([
    [0.0, 1.0],
    [1 / (1 + r), r / (1 + r)],
    [(1 + r) / (1 + 2 * r)] * 2,
])
np.testing.assert_allclose(
    full_output, hand_output, rtol=0, atol=1e-12,
)
assert np.all(full_weights[~allowed] == 0)
print("Allowed mask, with 1 for an allowed read")
print(allowed.astype(int))
print("Full causal attention weights")
with np.printoptions(precision=9, suppress=True):
    print(full_weights)
```

OUTPUT

```
Allowed mask, with 1 for an allowed read
[[1 0 0]
 [1 1 0]
 [1 1 1]]
Full causal attention weights
[[1.          0.          0.         ]
 [0.330238451 0.669761549 0.         ]
 [0.248255078 0.248255078 0.503489843]]
```

Now feed the tokens one at a time. The cache starts empty, and each call appends the incoming key and value before computing attention. The first call sees only position 1. The second sees positions 1 and 2, and the third sees all three. Thus each streaming query sees exactly the entries its row of the full mask allows. Check the contents of both cached arrays after every call, and compare the last entry of each trace with the corresponding full-pass output.

```
attention_only = ("attention",)
attention_stores = initial_stores(attention_only)
stream_outputs = []
print("Position  Key shape  Value shape")

for position, token in enumerate(cache_tokens, start=1):
    token_trace, attention_stores = stack_step(
        token, attention_only, attention_stores,
    )
    stream_outputs.append(token_trace[-1])
    keys, values = attention_stores[0]
    np.testing.assert_array_equal(keys, full_keys[:position])
    np.testing.assert_array_equal(values, full_values[:position])
    print(f"{position:>8}  {str(keys.shape):>9}  {values.shape}")

stream_output = np.stack(stream_outputs)
np.testing.assert_allclose(
    stream_output, full_output, rtol=0, atol=1e-12,
)
print("Full pass and streaming outputs, respectively")
with np.printoptions(precision=9, suppress=True):
    print(full_output)
    print(stream_output)
print("Maximum absolute difference",
      f"{np.max(np.abs(stream_output - full_output)):.3e}")
```

OUTPUT

```
Position  Key shape  Value shape
       1     (1, 2)  (1, 2)
       2     (2, 2)  (2, 2)
       3     (3, 2)  (3, 2)
Full pass and streaming outputs, respectively
[[0.          1.         ]
 [0.669761549 0.330238451]
 [0.751744922 0.751744922]]
[[0.          1.         ]
 [0.669761549 0.330238451]
 [0.751744922 0.751744922]]
Maximum absolute difference 0.000e+00
```

The two methods give the same three output rows within the float64 absolute tolerance 10^{-12} with zero relative tolerance. The stored keys and values match the full-pass projections exactly at every prefix. Equality follows because each query uses the same projection, scaling, and ordered set of visible keys and values. In the streaming pass that set is defined by the prefix already in the cache. In the full pass it is defined by the Boolean mask.